



- O teste tem a duração de 2h

1. [1.5] Descreva o que entende por processo e por *process control block*.
2. [1.5] Explique como é feita a comutação entre processos ao nível do processador.
3. [1.5] Explique as técnicas base de comunicação entre processos, indicando possíveis vantagens e inconvenientes de cada.
4. Considere a seguinte *shell script* (bash)

```
case $1 in *.bak
    echo "$1: nome não válido"; exit 1;;
esac

tmp=/tmp/tmp$$

cp $1 $tmp 2>/dev/null
vi $tmp

if cmp $1 $tmp
then
    rm $tmp
else
    mv $1 $1.bak 2>/dev/null
    mv $tmp $1
fi
```

- a) [2.0] Diga qual a finalidade da *shell script* apresentada e explique detalhadamente o seu funcionamento.

5. Considere o seguinte programa escrito em python

```
import os
print "Sou o pai: " + str(os.getpid())
pid_filho = os.fork()

if pid_filho == 0:
    print "Sou o filho1: " + str(os.getpid())
else:
    pid_filho2 = os.fork()
    if pid_filho2 == 0: #filho
        print "Sou o Filho2: " + str(os.getpid())
```

- a) [1.5] Quantos processos são criados por este programa? Justifique desenhando a respetiva árvore de processos.
6. [2.0] Descreva sucintamente como pode ser resolvido o problema da secção crítica e quais as propriedades que deverão ser respeitadas.
7. [1.5] O que entende por fragmentação de memória? Descreva sucintamente os vários tipos de fragmentação de memória que conhece.
8. [1.0] Explique detalhadamente como poderá ser feita a tradução de endereços, usando memória paginada, para o caso de endereços virtuais de 32 bits, com páginas de 4 KB.
9. Considere o seguinte programa escrito em Python

```

from multiprocessing import Process, Value, Lock
global = Value("i",0)
mutex = Lock()

def incrementa():
    mutex.acquire()
    global.value += 1
    mutex.release()

processos=[]

for i in range(100):
    NewP = Process(target=incrementa)
    NewP.start()
for i in range(100):
    NewP.join()
print global.value

```

- b) [1.0] Que tipo de comunicação entre processos é usado neste programa? Justifique.
- c) [1.0] Qual o resultado de execução após quatro invocações deste programa? Se não for sempre o mesmo valor, altere o código para que isso passe a acontecer.

10. [2.0] Suponha que há um acesso à memória paginada e a página em causa não está na memória principal. Descreva a sequência de eventos associadas a esta ação.
11. [2.0] Suponha que tem um sistema operativo no qual as páginas apenas são movidas da memória de massa para a memória principal após um *page fault* e que usa um algoritmo de substituição de páginas do tipo FIFO e que, a cada momento, apenas 5 páginas por processo podem estar na memória principal. Indique justificadamente quantos *pages faults* ocorrem quando um dado processo tem a seguinte sequência de acesso a páginas de memória: 1,0,1,0,1,0,1,7,2,7,3,0,7,2,3,0,3,11,3,2,1,2,0,1,7,0,1,0,0,0,5
12. [1.5] Explique porque é que, no âmbito da virtualização de sistemas operativos, o Virtual Machine Manager (*hypervisor*) tem que usar técnicas de “*trap and emulate*” ou de “*binary translation*”?